

14장

실전 2D 게임 프로젝트

학습목표

- 2D 타워 디펜스 게임의 구조를 이해하고, 프로젝트를 구성할 수 있다.
- 유니티 타일 맵과 맵 시스템을 이해하며, 2D 타일 맵을 구성할 수 있다.
- NavMesh 기반 길 찾기를 이해하고, 적 NPC의 생성과 이동을 구현할 수 있다.
- 공격 무기 시스템을 이해하고, 전투 로직과 게임 관리를 구현할 수 있다.

목차

1. 2D 타워 디펜스 게임
2. 유니티 타일 맵과 맵 시스템
3. 2D NavMesh 기반 길 찾기와 적 NPC 생성 및 파괴
4. 공격 무기 시스템

01
2D 타워 디펜스 게임

01 2D 타워 디펜스 게임

- 입문자는 2D 게임에 비교적 쉽게 접근 가능
- 타워 디펜스 게임
  - 적이 정해진 경로에 따라 움직이고, 플레이어가 공격 무기로 적을 파괴



(a) 〈플랜츠 vs. 좀비〉



(b) 〈킹덤 러시〉

그림 14-1 타워 디펜스 게임

01 2D 타워 디펜스 게임

- 맵 시스템
  - 일반적으로 타일 맵 이용
  - 적의 이동 경로 길이를 통해 난이도 조절
- 웨이브 로직
  - 일정 주기마다 적을 생성
- 길 찾기
  - 적에 인공지능 길 찾기를 적용
- 공격 메커니즘
  - 적의 이동 경로에 플레이어의 공격 무기를 배치하여 적을 공격
- 업그레이드 시스템
  - 적을 처치하여 재화를 얻고, 재화를 사용하여 공격 무기를 업그레이드
- 스테이지 클리어 로직
  - 목표 지점 방어에 성공하면 해당 스테이지 통과

02

유니티 타일 맵과 맵 시스템

02 유니티 타일 맵과 맵 시스템

1. 맵 시스템

- 통맵
- 타일 맵



(a) 통맵



(b) 타일 맵

그림 14-2 통맵과 타일 맵

02 유니티 타일 맵과 맵 시스템

2. 그래픽 리소스 준비

01 ‘2DDefenceGame’ 프로젝트를 생성하고,
씬 이름 ‘Main2DGameScene’으로 변경한 후 Assets 폴더 하위의 Scenes 폴더에 저장.

02 [Window]-[Asset Store]를 선택하여 에셋 스토어에 접속하고, ‘Pixel Art Top Down - Basic’을 검색하여 에셋을 선택하고 <내 에셋에 추가하기> 클릭.

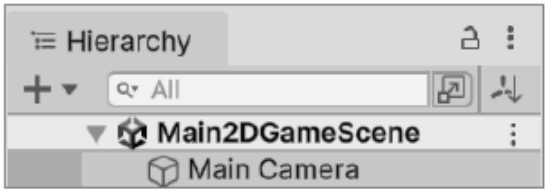


그림 14-3 메인 게임 씬 설정

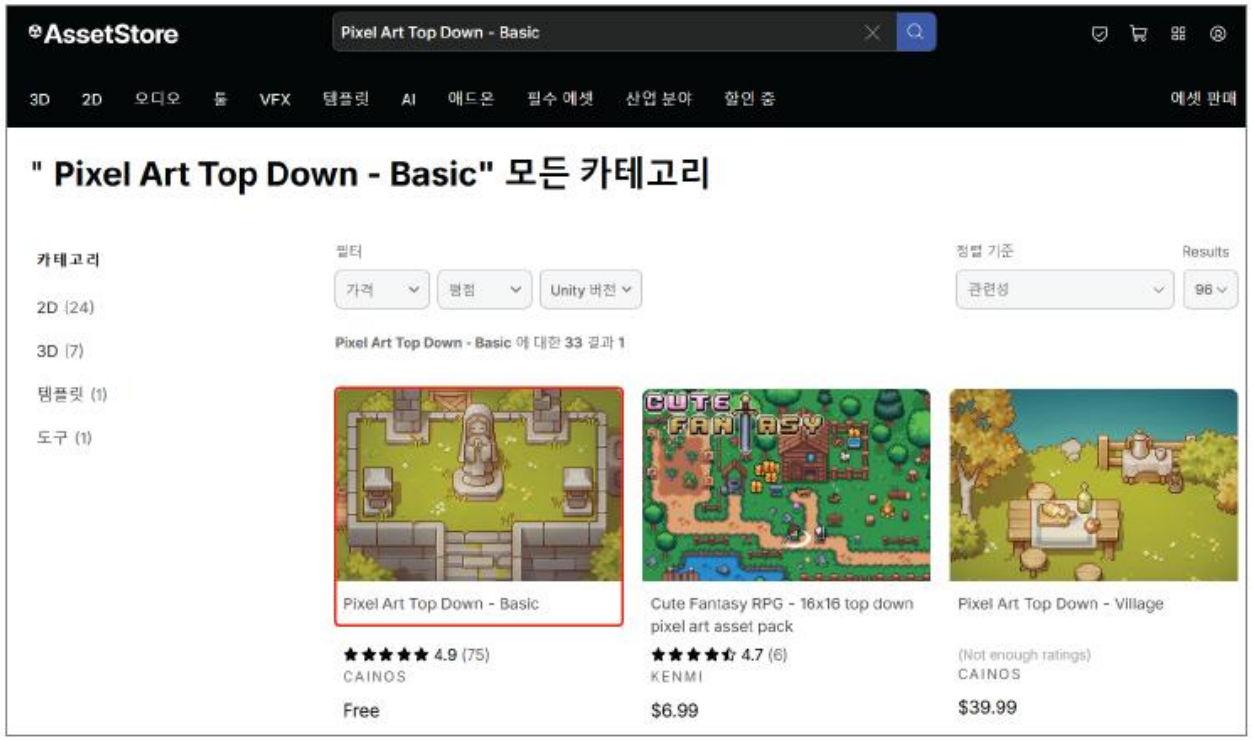


그림 14-4 맵 타일에 사용할 에셋 검색

2. 그래픽 리소스 준비

03 [Window]-[My Assets]에서 에셋을 PC에 다운로드하고,
<Import 버전 번호 to project>를 클릭하여 현재 프로젝트에 에셋을 임포트.

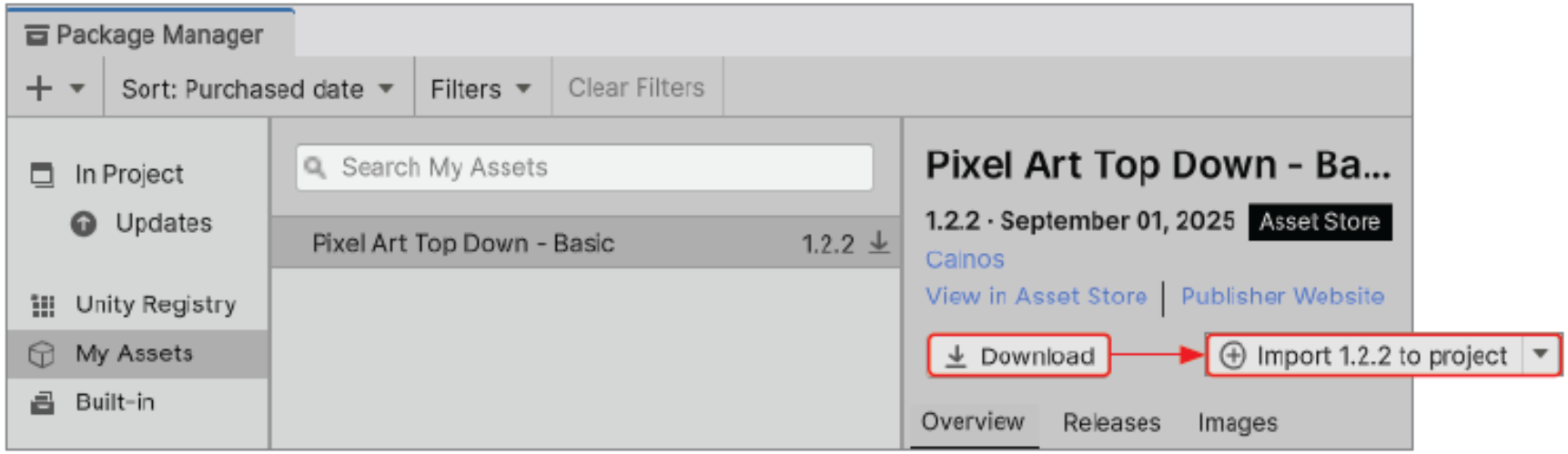


그림 14-5 Package Manager에서 에셋 임포트

2. 그래픽 리소스 준비

04 Assets 폴더 아래에 Cainos라는 폴더가 생성되며, 이 안에 리소스가 들어 있음. Texture 폴더에는 그래픽 리소스가 들어 있음.

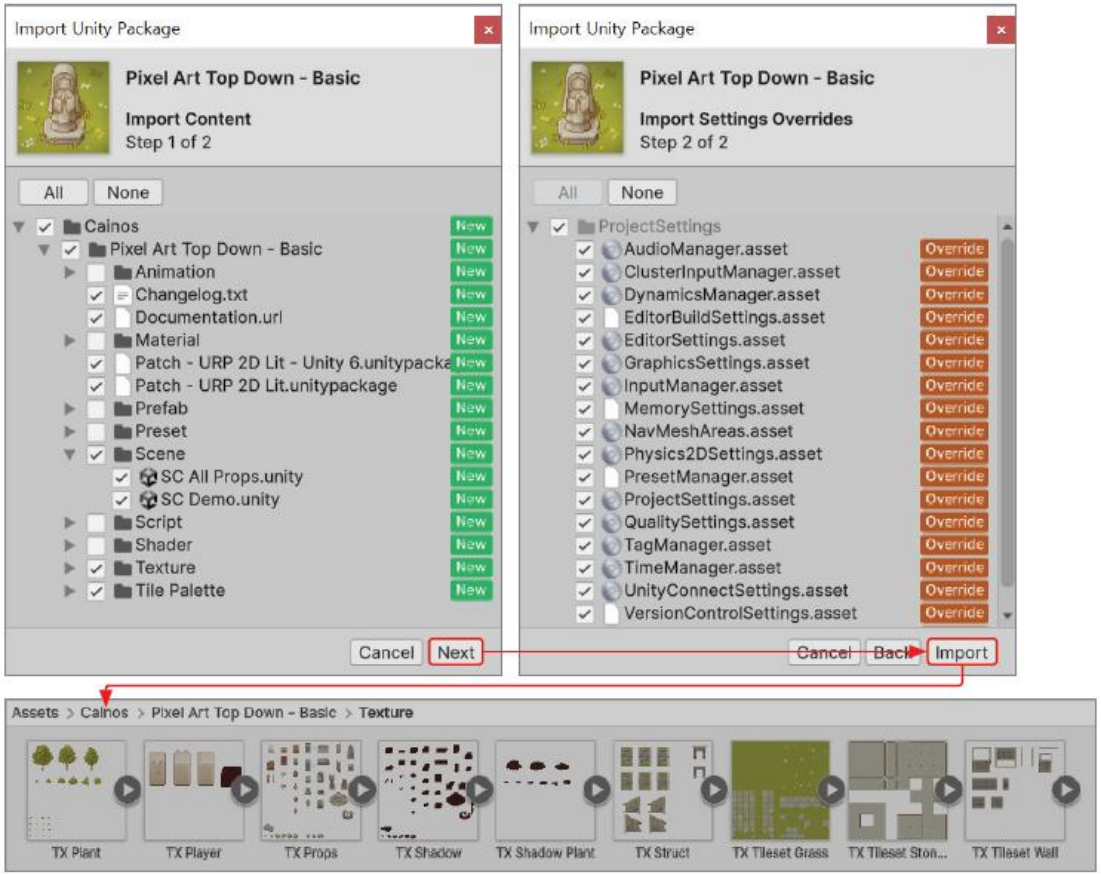


그림 14-6 Pixel Art Top Down - Basic 에셋 폴더의 구성

3. 2D 타일 맵 구성

- 타일 맵을 이용하여 이미지로 타일을 생성하고 빠르게 전체 맵을 구성할 수 있음.

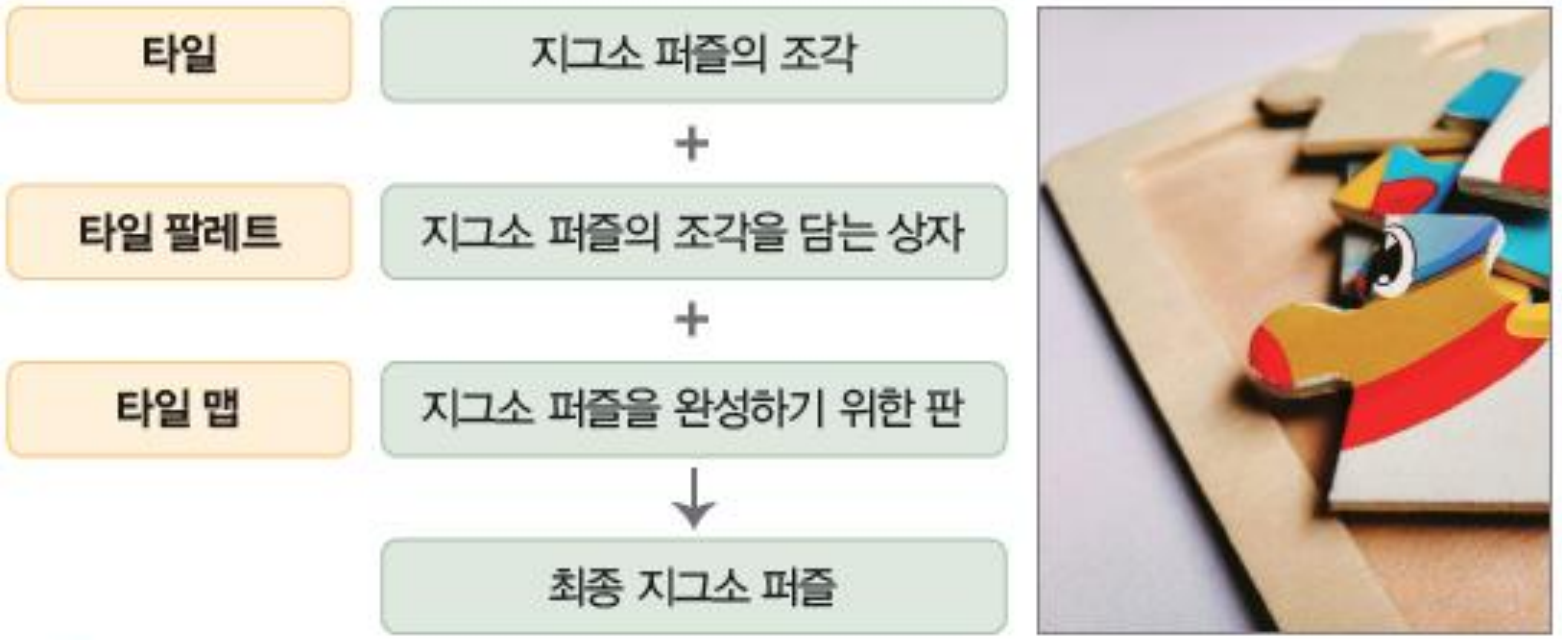
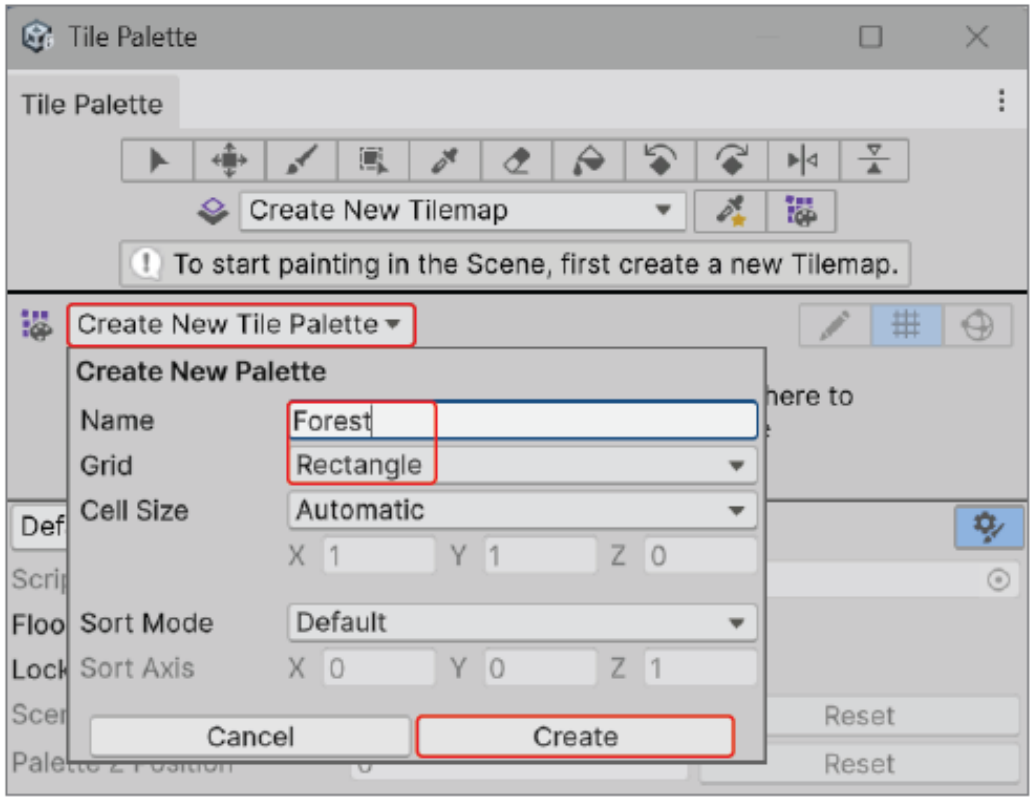


그림 14-7 타일, 타일 팔레트, 타일 맵의 관계

3. 2D 타일 맵 구성

◆타일 팔레트 생성

- [Window]-[2D]-[Tile Palette]를 선택하여 타일 팔레트 창을 열고, <Create New Tile Palette>를 클릭하여 타일 팔레트 'Forest'를 생성. Grid 속성을 Rectangle로 설정한 후 Assets 폴더 하위에 TileSet 폴더를 생성하고 타일 팔레트를 저장.



3. 2D 타일 맵 구성

◆타일의 종류

- 일반 타일
- Animated Tile
- Rule Tile
- Isometric Rule Tile
- Hexagonal Rule Tile
- Rule Override Tile
- Advanced Rule Override Tile
- Custom Rule Tile

3. 2D 타일 맵 구성

◆일반 타일 생성

- 타일 이미지를 개별적으로 사용하거나, 타일 셋을 사용
- 타일 셋: 여러 타일 이미지를 하나의 이미지로 합친 것. 여러 타일 이미지를 효율적으로 관리하고 메모리 사용을 최적화할 수 있음

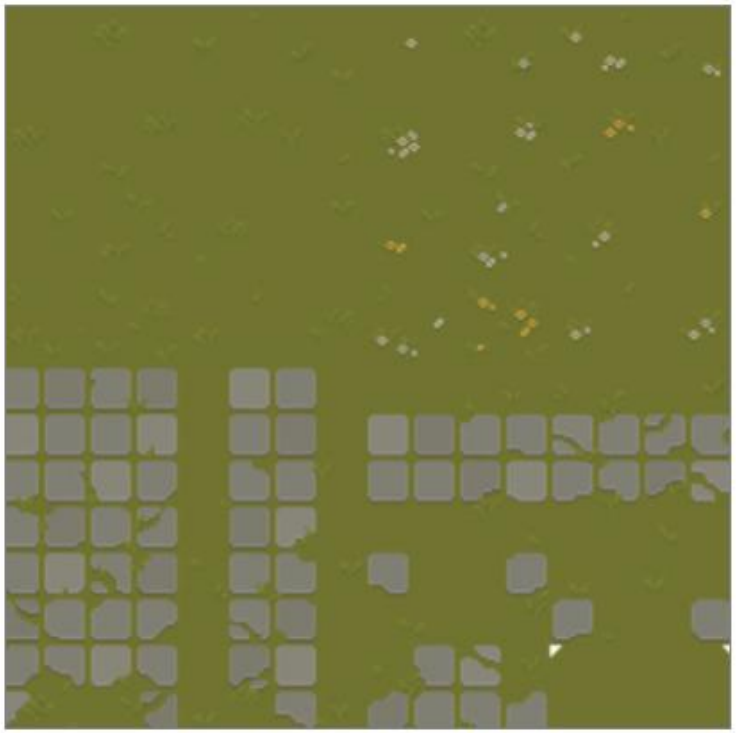


그림 14-9 Pixel Art Top Down - Basic 에셋의 타일 셋

3. 2D 타일 맵 구성

◆일반 타일 생성

01 Pixel Art Top Down – Basic 에셋의 Texture 폴더에 있는 TX Tileset Grass.png를 [Assets]-[Tile] 폴더에 복사



그림 14-10 Pixel Art Top Down – Basic 에셋의 타일 셋 이미지

3. 2D 타일 맵 구성

◆일반 타일 생성

02 이미지의 Texture Type이 Sprite (2D and UI)로 설정되었는지 확인

03 Sprite Mode를 Multiple로 변경하고, Pixel Per Unit를 32로 설정

04 Filter Mode를 Point (no filter)로 설정하여 이미지 흐려짐과 왜곡을 방지

05 Format을 RGBA 32 bit로 설정

06 설정을 완료한 후 <Open Sprite Editor>를 클릭

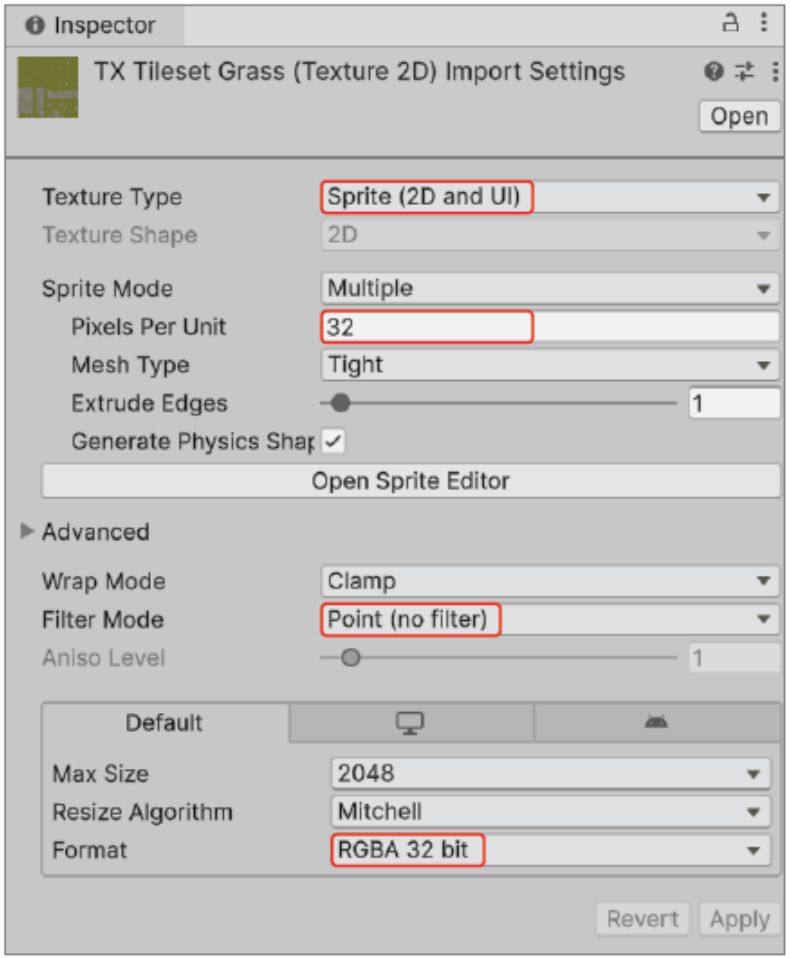


그림 14-11 TX Tileset Grass 이미지 속성 설정

02 유니티 타일 맵과 맵 시스템

07 Type은 Grid By Cell Size로, Pixel Size는 X와 Y를 32로 설정한 후 <Slice>를 클릭

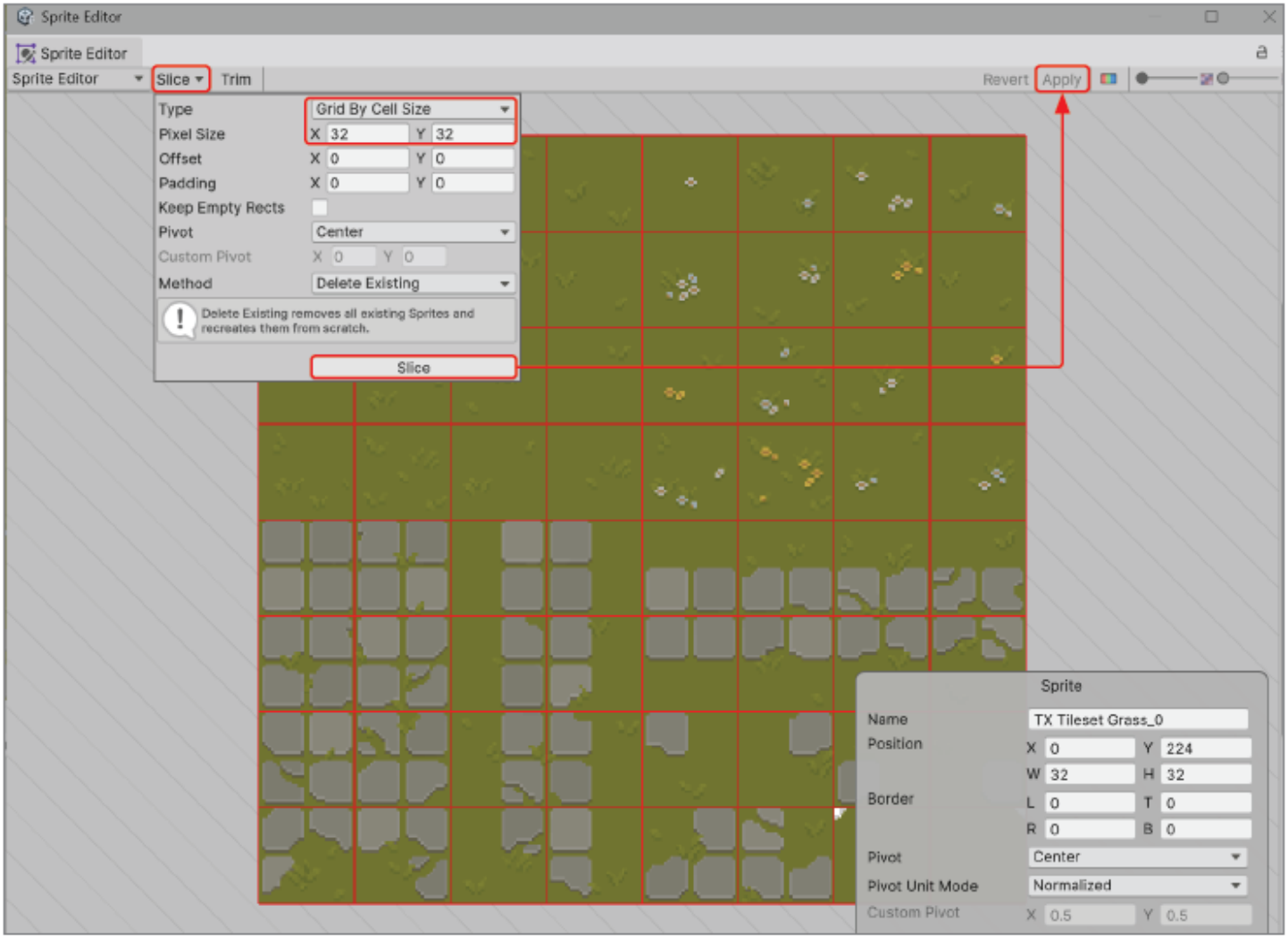


그림 14-12 Sprite Editor에서 32×32픽셀 타일 이미지 생성

3. 2D 타일 맵 구성

◆Animated Tile 생성

- [Create]-[2D]-[Tiles]-[Animated Tile]을 선택하여 Animated Tile을 생성

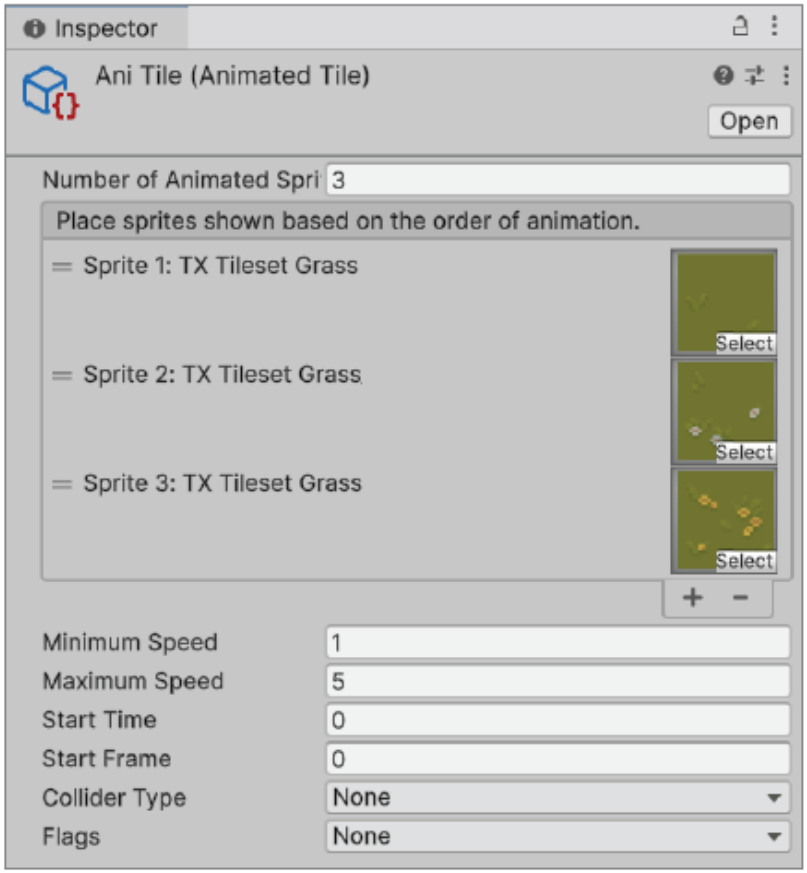


그림 14-13 Animated Tile의 속성

3. 2D 타일 맵 구성

◆타일 팔레트에 타일 등록

- 스프라이트를 타일 팔레트에 드래그하거나(일반 타일)
  생성된 타일을 타일 팔레트에 드래그하여(일반 타일 외 타일) 등록

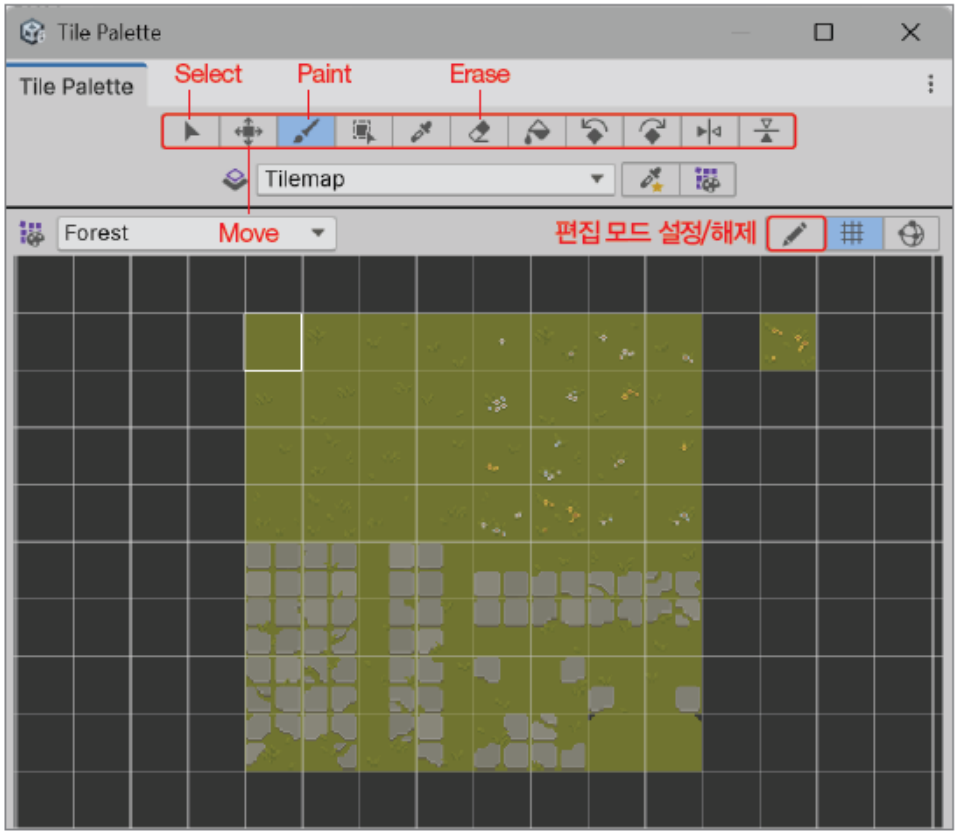


그림 14-14 타일 팔레트에 등록된 타일과 메뉴 아이콘

3. 2D 타일 맵 구성

◆타일 맵 생성 및 그리기

01 Hierarchy 윈도우에서 [2D Object]-[Tilemap]-[Rectangular]를 선택하여 타일 맵을 생성하면 Grid 오브젝트와 그 자식으로 Tilemap 오브젝트가 생성됨.

02 Grid 오브젝트의 자식으로 Tilemap을 하나 더 생성하고, 기존 타일 맵은 BgTilemap, 새 타일 맵은 ObjTilemap으로 이름 변경.

03 ObjTilemap의 Tilemap Renderer 속성 중에서 Order In Layer를 1로 설정.

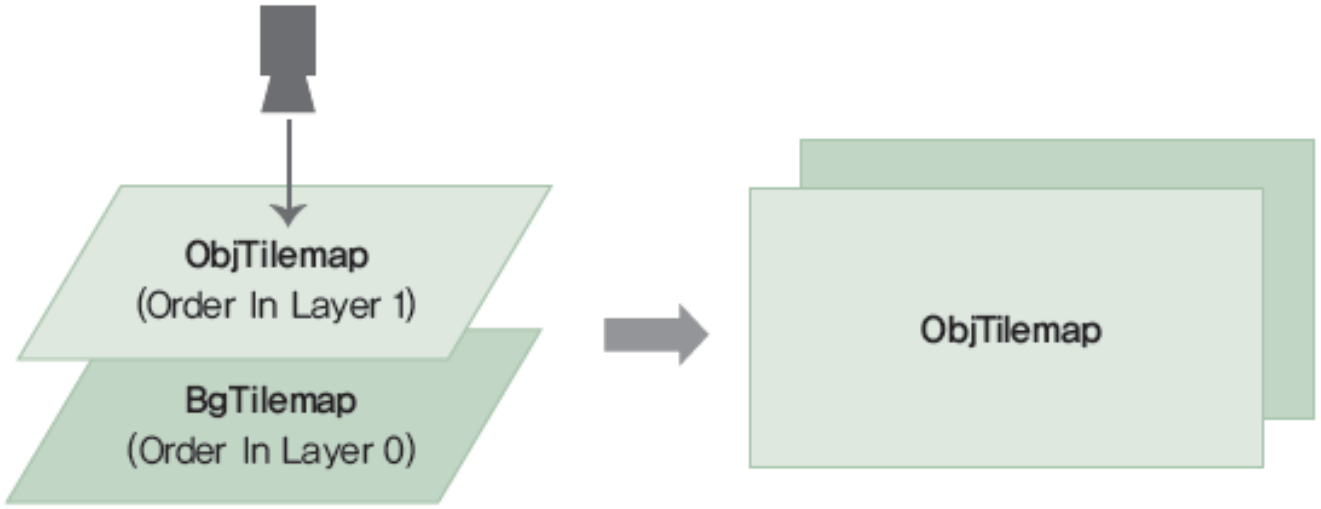


그림 14-15 Order In Layer의 역할

3. 2D 타일 맵 구성

◆타일 맵 생성 및 그리기

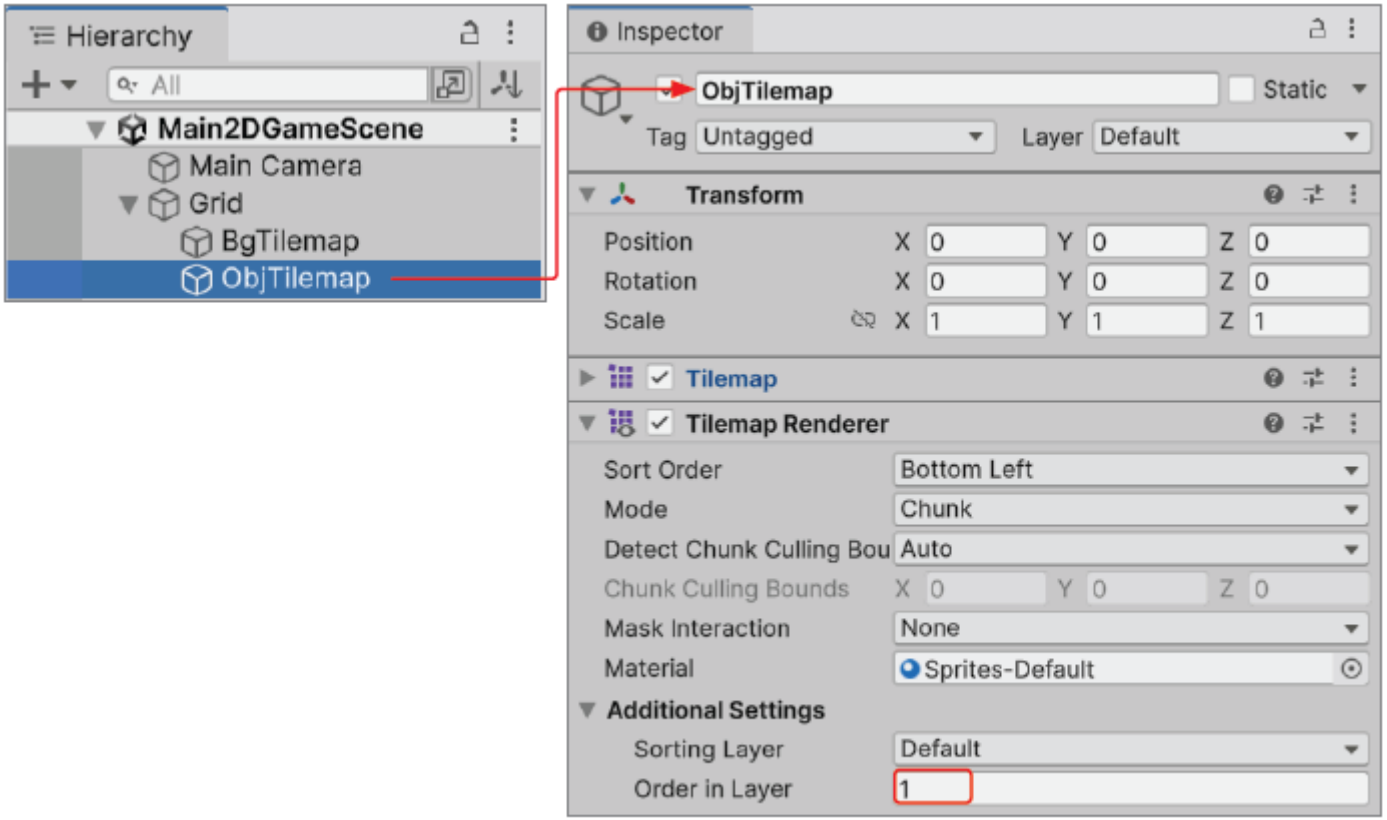


그림 14-16 ObjTilemap의 설정

3. 2D 타일 맵 구성

◆타일 맵 생성 및 그리기

04 타일 팔레트를 열어 Active Target 속성을 BgTilemap으로 설정하고, 메뉴에서 Paint를 선택해 배경 타일을 그림. 타일 삭제는 Erase.

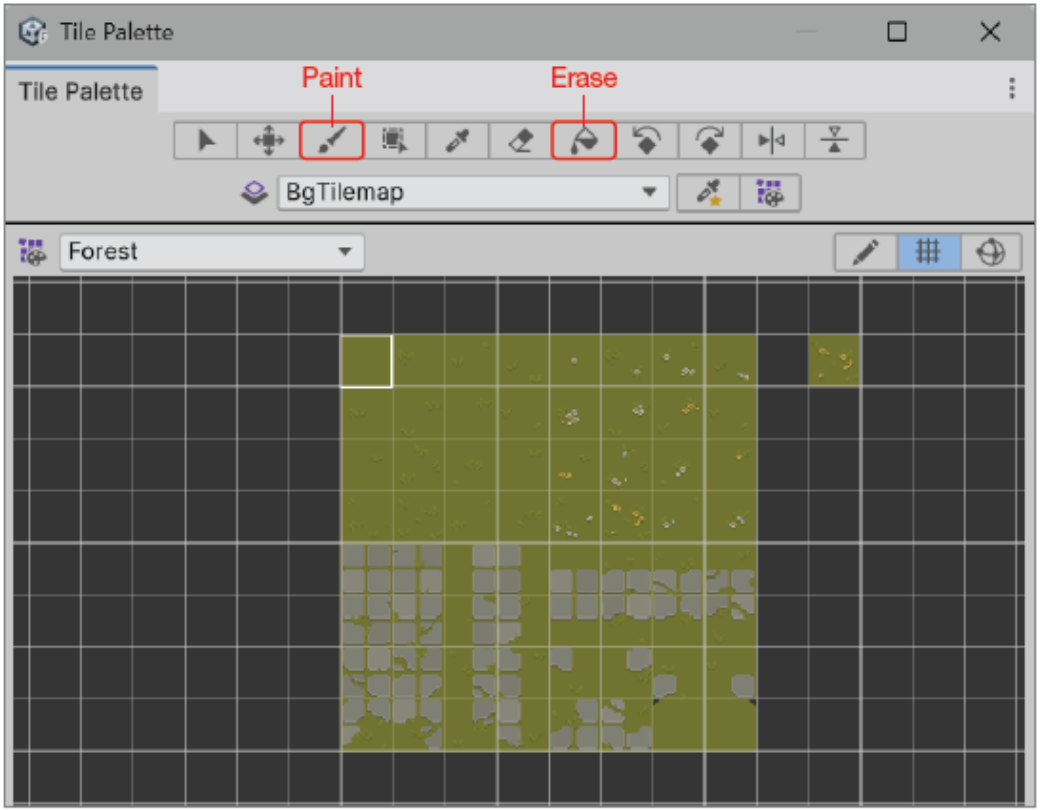


그림 14-17 타일 팔레트에서 타일을 그리기 위한 선택 설정

02 유니티 타일 맵과 맵 시스템

- Active Target을 BgTilemap으로 설정하면 타일이 배경 레이어에 그려지고, ObjTilemap으로 설정하면 타일이 오브젝트 레이어에 그려짐.

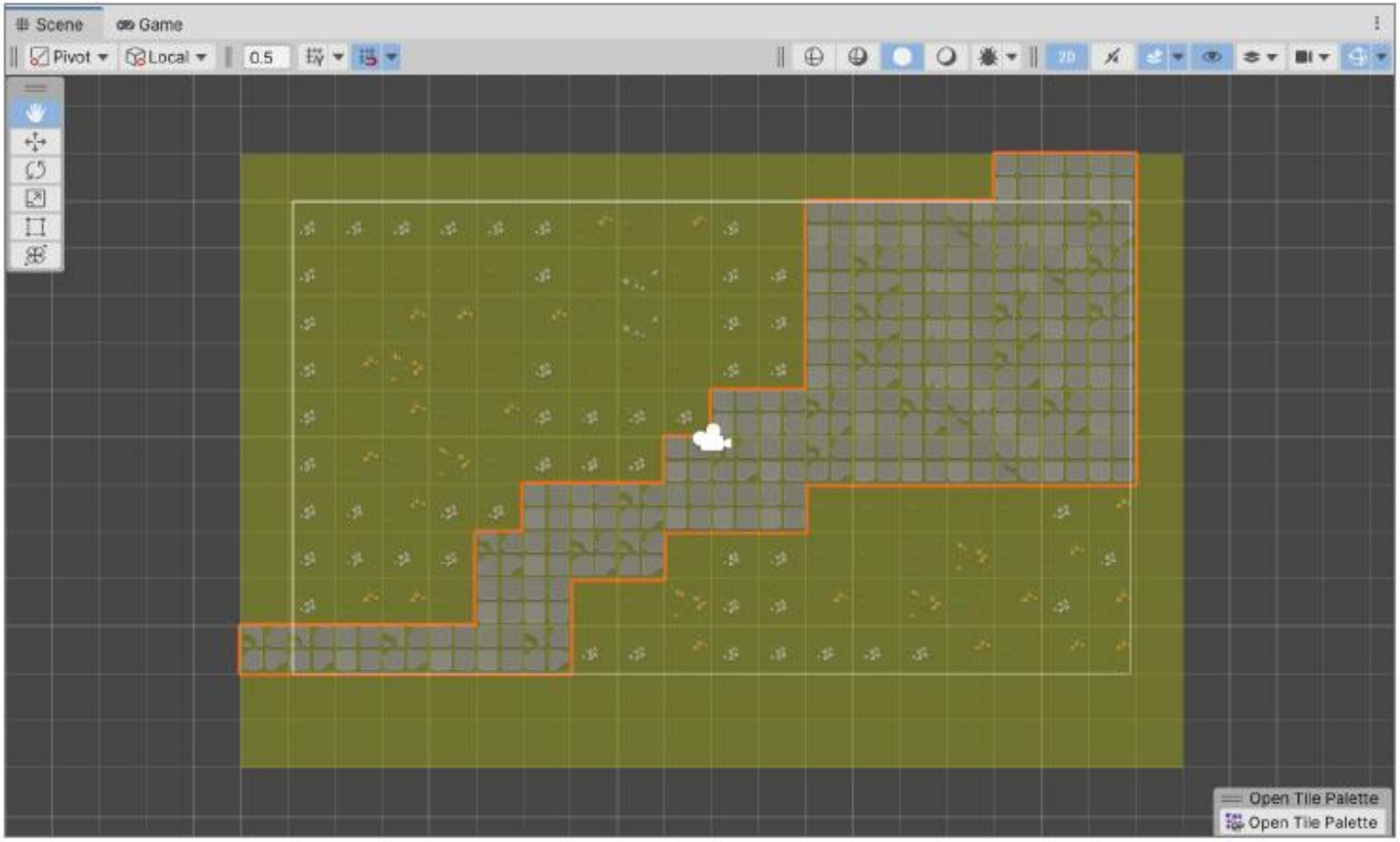


그림 14-18 타일 맵 완성

03
2D NavMesh 기반 길 찾기와 적 NPC 생성 및 파괴

03 2D NavMesh 기반 길 찾기와 적 NPC 생성 및 파괴

1. 적 길 찾기

◆2D 내비게이션 메시 생성

01 2D NavMesh 커스텀 패키지 다운로드 전에, https://git-scm.com/install/windows에 접속하여 윈도우 버전 git을 설치

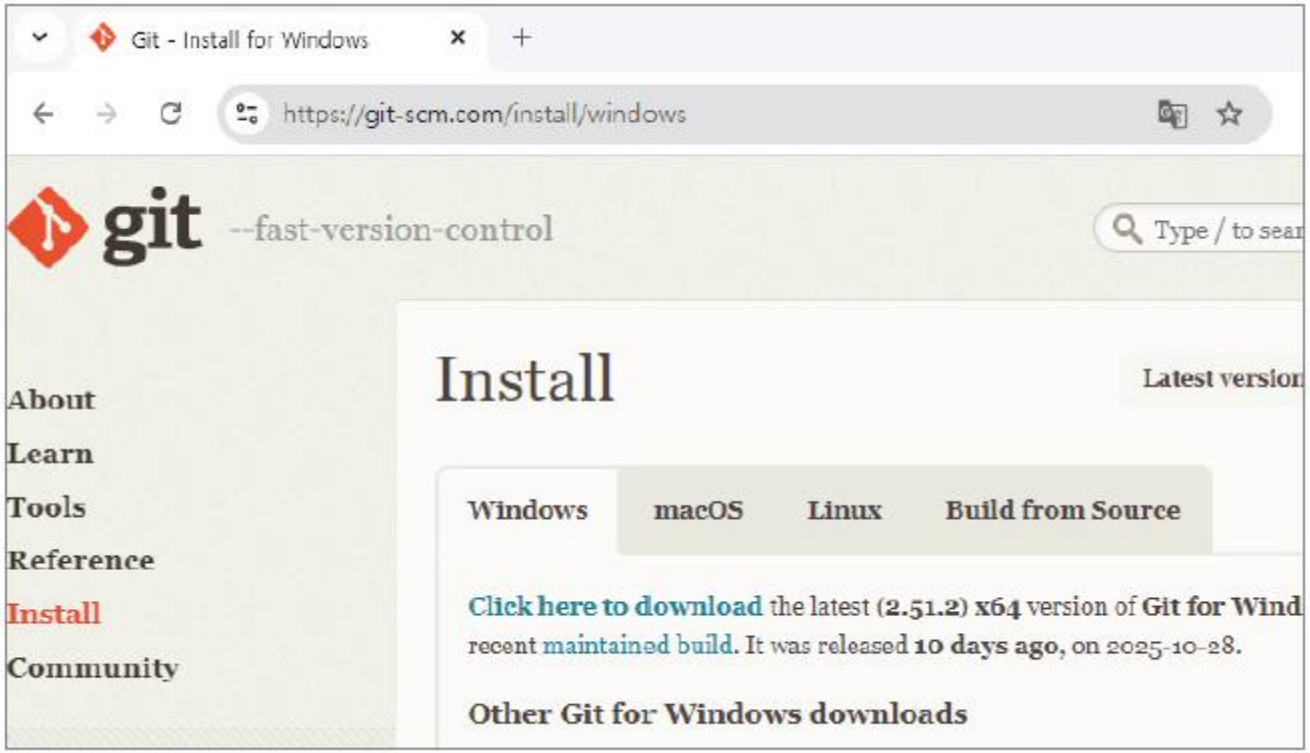


그림 14-19 윈도우 버전 git 설치

1. 적 길 찾기

◆2D 내비게이션 메시 생성

02 Package Manager 상단의 [+]-[Install package from git URL]을 선택하고 URL(https://github.com/h8man/NavMeshPlus.git)을 입력한 후 <Install> 클릭.

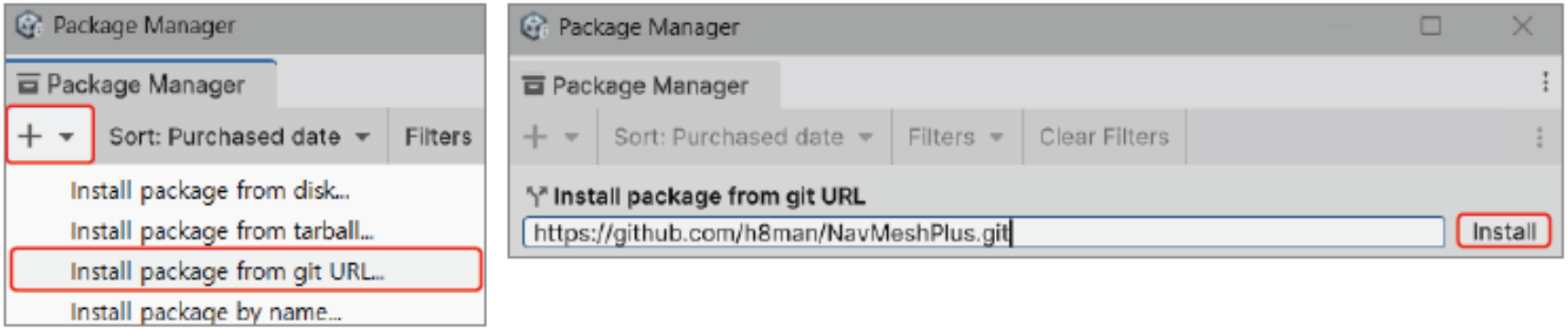


그림 14-20 2D NavMesh 커스텀 패키지 설치

03 2D NavMesh 기반 길 찾기와 적 NPC 생성 및 파괴

1. 적 길 찾기

◆2D 내비게이션 메시 생성

03 Package Manager에서 [Unity Registry]를 선택하고 AI Navigation을 찾아 설치.

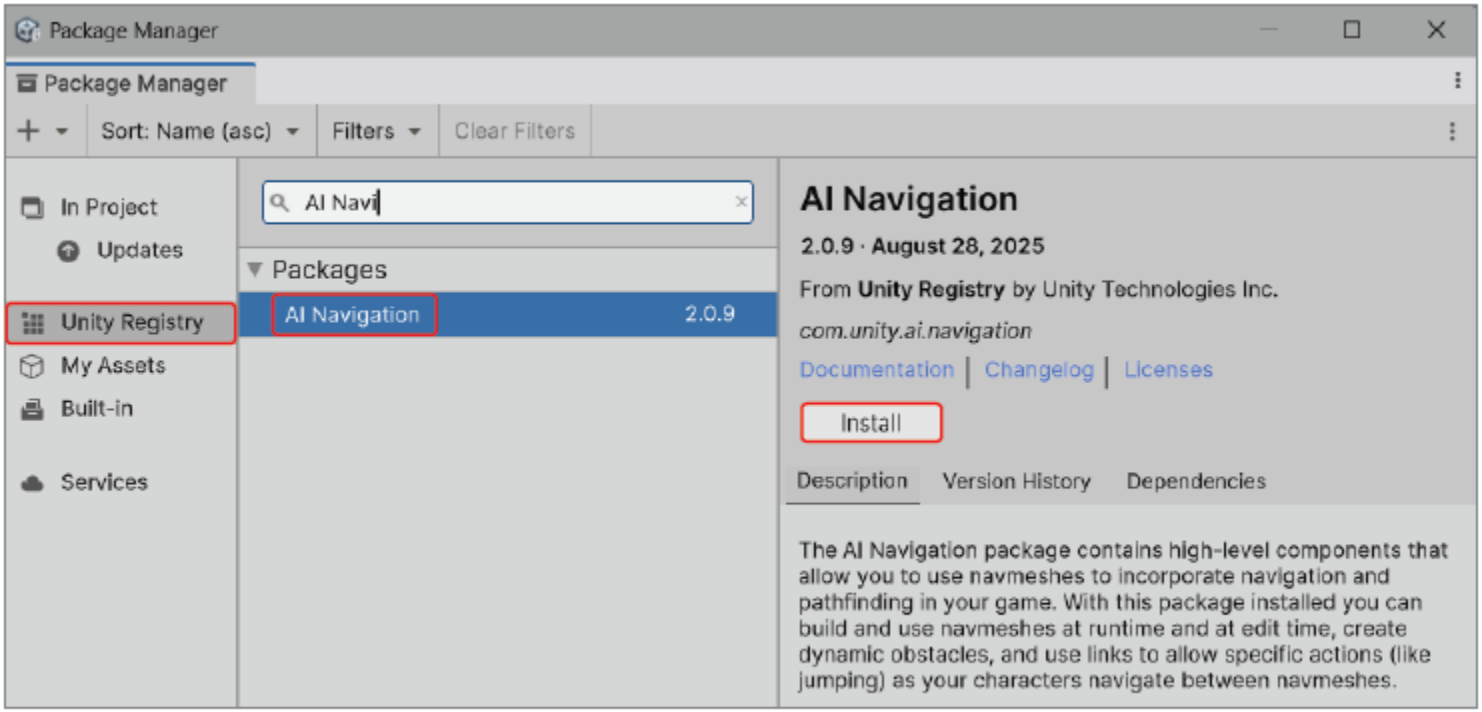


그림 14-21 AI Navigation 설치

1. 적 길 찾기

◆2D 내비게이션 메시 생성

04 BgTilemap 오브젝트를 선택한 후 Navigation Modifier를 컴포넌트로 추가. Override Area 속성을 활성화하고 Area를 Not Walkable로 설정.

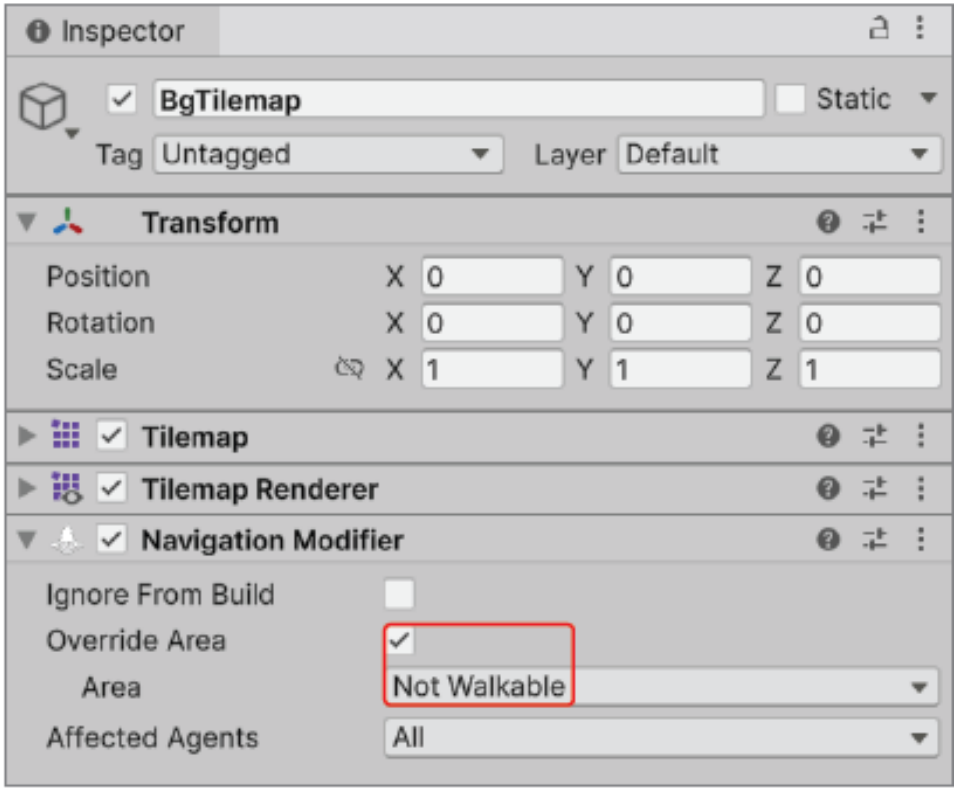


그림 14-22 BgTilemap의 Navigation Modifier 설정

03 2D NavMesh 기반 길 찾기와 적 NPC 생성 및 파괴

1. 적 길 찾기

◆2D 내비게이션 메시 생성

05 ObjTilemap 오브젝트에도 Navigation Modifier를 컴포넌트로 추가. Override Area 속성을 활성화하고 Area를 Walkable로 설정.

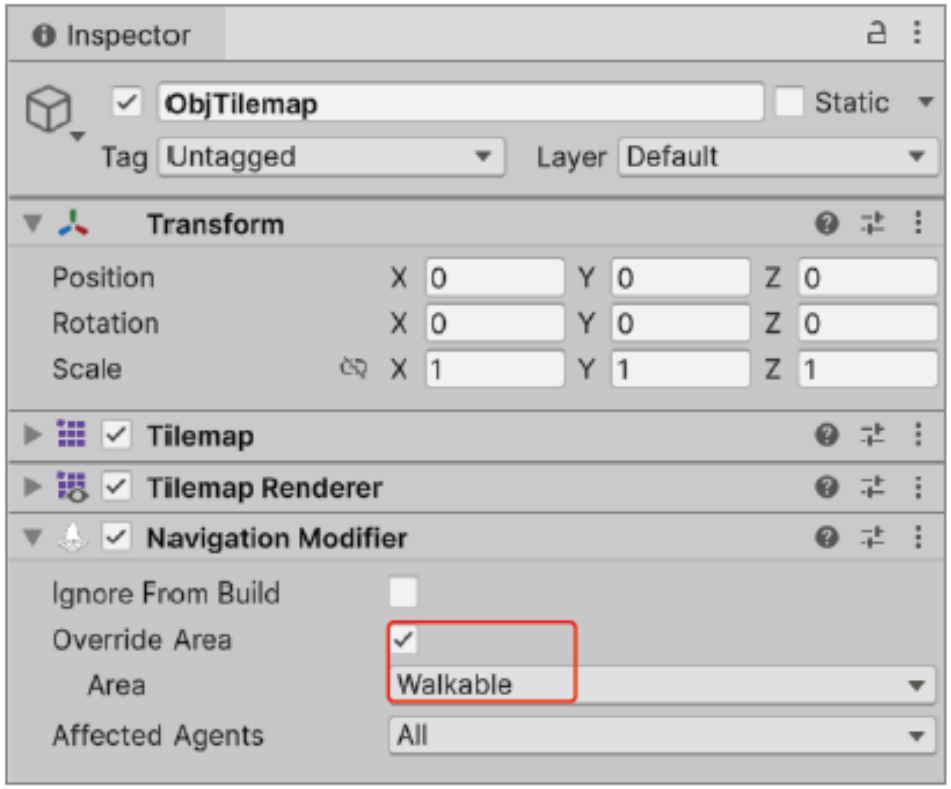


그림 14-23 ObjTilemap Navigation Modifier 설정

03 2D NavMesh 기반 길 찾기와 적 NPC 생성 및 파괴

1. 적 길 찾기

◆2D 내비게이션 메시 생성

06 빈 오브젝트 'NavMeshMgr'에 Navigation Surface를 컴포넌트로 추가.

07 NavMeshMgr 오브젝트의 컴포넌트로 Navigation CollectSources2d를 추가하고 <Rotate Surface to XY>를 클릭

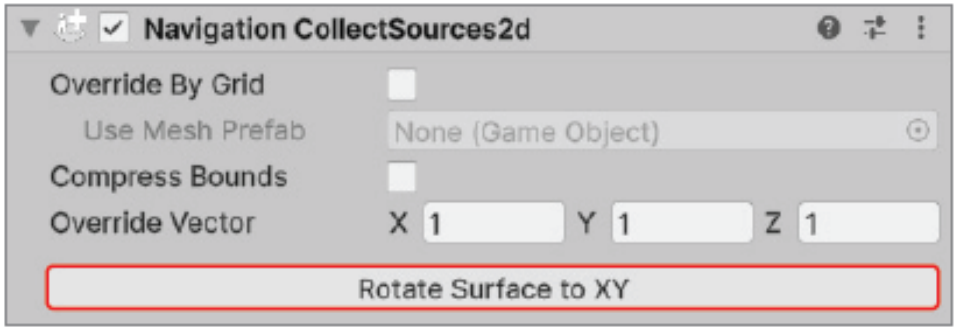


그림 14-24 Navigation CollectSources2d 설정

03 2D NavMesh 기반 길 찾기와 적 NPC 생성 및 파괴

1. 적 길 찾기

◆2D 내비게이션 메시 생성

08 모든 타일 맵에 Navigation Modifier를 컴포넌트로 추가하고 Override Area 속성을 활성화. NavMeshMgr 오브젝트의 컴포넌트인 Navigation Surface의 <Bake>를 클릭

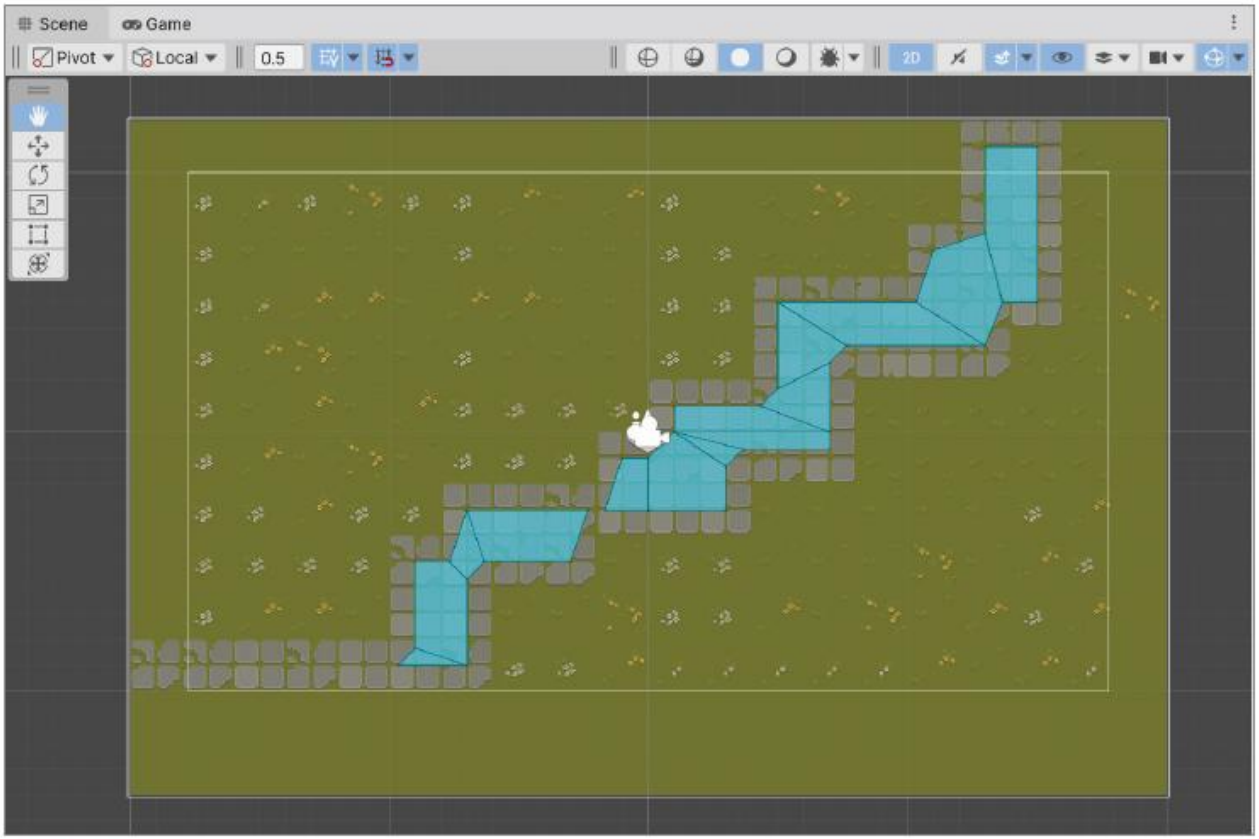


그림 14-25 Navigation Mesh 생성

1. 적 길 찾기

◆2D 내비게이션 메시 생성

09 [Window]-[AI]-[Navigation]을 선택하고 Agents의 Radius를 0.2로 수정.

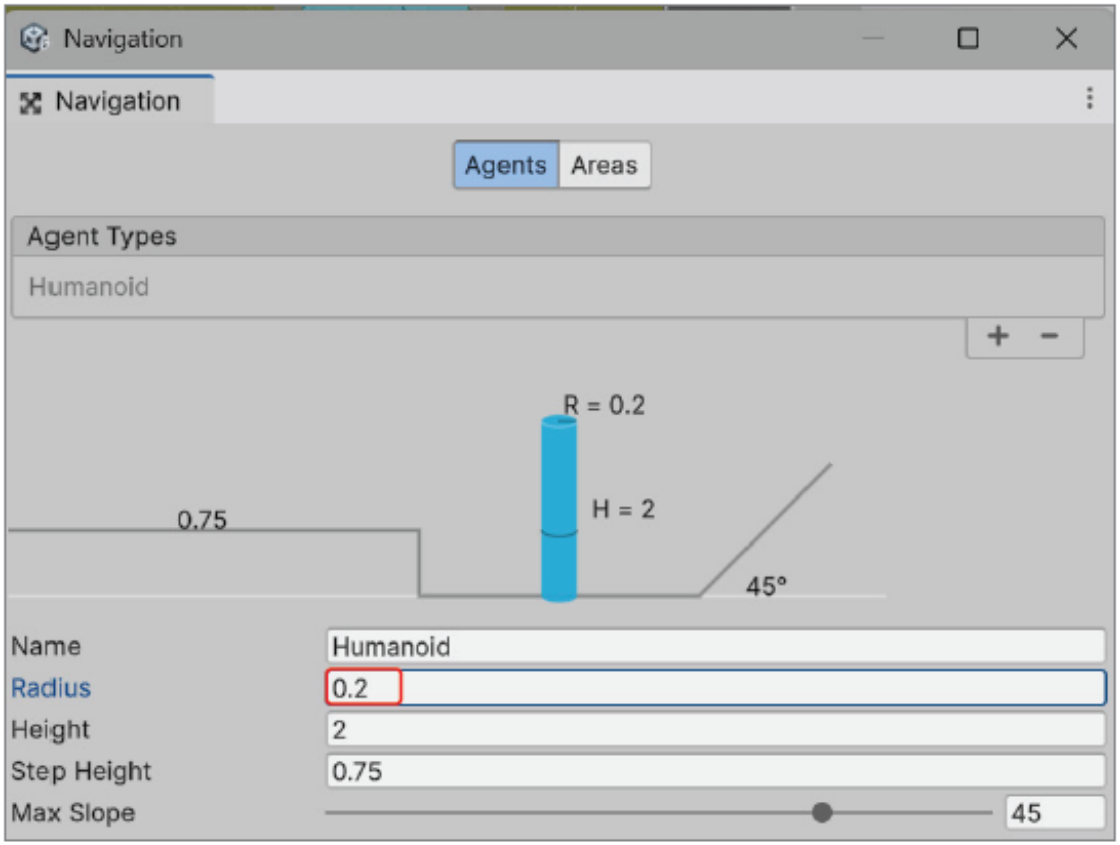


그림 14-26 Agents의 Radius 수정

03 2D NavMesh 기반 길 찾기와 적 NPC 생성 및 파괴

1. 적 길 찾기

◆2D 내비게이션 메시 생성

10 NavMeshMgr 오브젝트의 컴포넌트인 Navigation Surface에서 <Bake>를 클릭.

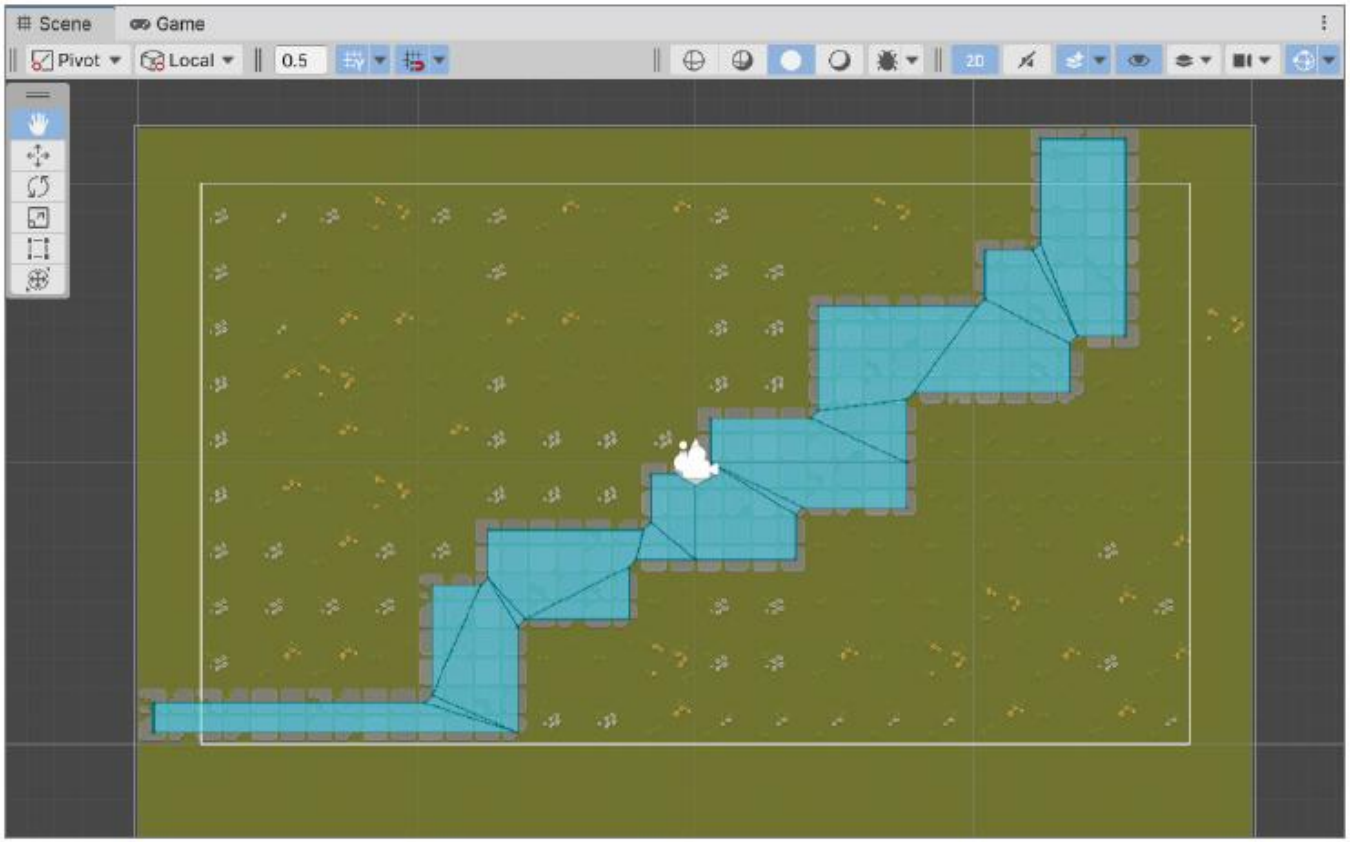


그림 14-27 Navigation Mesh 재생성

03 2D NavMesh 기반 길 찾기와 적 NPC 생성 및 파괴

1. 적 길 찾기

◆길 찾기 확인

01 [2D Object]-[Sprite]-[Circle]로 Circle 오브젝트를 생성하고 이름을 'Enemy'로 지정. 위치를 (-8.5, -4.5, 0), 크기를 (0.5, 0.5, 0.5)로 설정.

02 Sprite Renderer의 Order in Layer 속성을 2로 설정

03 Enemy 오브젝트에 NavMesh Agent 컴포넌트를 추가 후 Speed를 2로 설정.

04 목표 지점을 향해 움직이는 로직을 구현할 C# 스크립트 파일 'ForceNavMove' 생성.

03 2D NavMesh 기반 길 찾기와 적 NPC 생성 및 파괴

1. 적 길 찾기

◆길 찾기 확인

05 ForceNavMove.cs 파일에 [코드 14-1]과 같이 작성하여 Enemy의 컴포넌트로 추가.

코드 14-1 적의 자동 이동 — ForceNavMove.cs

```
01  using UnityEngine;
02  using UnityEngine.AI;
03  public class ForceNavMove : MonoBehaviour
04  {
05      public Transform endObject;
06      private NavMeshAgent enemyAgent;
07      private void Start()
08      {
09          enemyAgent = GetComponent<NavMeshAgent>();
10          enemyAgent.updateRotation = false;
11          enemyAgent.updateUpAxis = false;
12          enemyAgent.SetDestination(endObject.transform.position);
13      }
14  }
```

1. 적 길 찾기

◆길 찾기 확인

06 Enemy 오브젝트의 목표 지점으로 사용할 빈 오브젝트 'EndPoint'를 생성하고 위치를 (7.5, 5.5, 0)으로 설정

07 Enemy 오브젝트 Force Nav Move 속성의 End Object에 EndPoint 오브젝트를 설정

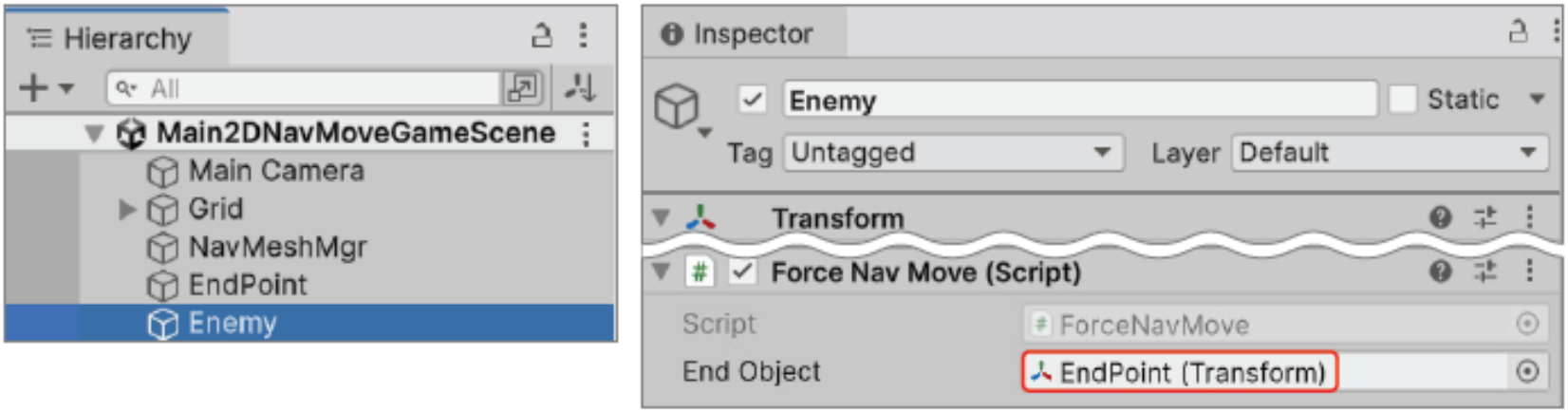


그림 14-28 Enemy 오브젝트의 설정

08 유니티 에디터의 실행 버튼을 클릭하고 Game 뷰에서 Enemy 오브젝트가 자동으로 이동하여 목표 지점에서 정지하는지 확인

2. 웨이브 로직과 적 생성 및 이동

◆적의 이동 로직 구현

01 [Assets]-[Scripts] 폴더에 C# 스크립트 파일 'Enemy'를 생성.

코드 14-2 적의 정보와 움직임 처리 Enemy.cs

```
01  public class Enemy : MonoBehaviour
02  {
03      [Header("적에 대한 속성 설정")]
04      public float hpValue = 100.0f;    // 적의 체력 설정
05
06      private EnemyMgr enemyMgr;
07
08      public void SetEnemeyMgr(EnemyMgr enemyMgr)
09      {
10          this.enemyMgr = enemyMgr;
11      }
12      public void OnDamage(float damageValue)
```

2. 웨이브 로직과 적 생성 및 이동

◆적의 이동 로직 구현

```
13      {
14          hpValue -= damageValue;
15          if (hpValue <= 0)
16          {
17              hpValue = 0;
18              enemyMgr.DieEnemy(gameObject);
19              Destroy(gameObject);
20          }
21      }
22  }
```

2. 웨이브 로직과 적 생성 및 이동

◆적의 이동 로직 구현

02 [2D Object]-[Sprite]-[Circle]로 오브젝트를 생성하고 이름을 'Enemy'로 지정.
위치를 (0, 0, 0), 크기를 (0.5, 0.5, 0.5)로 설정.
Sprite Renderer의 Order in Layer 속성은 2로, Color는 빨간색으로 설정.

03 NPC 오브젝트에 NavMesh Agent 컴포넌트를 추가하고, NavMesh Agent의 Speed를 1로 설정.

04 충돌 처리를 위해 Circle Collider 2D를 컴포넌트로 추가한 후 Radius을 0.5로 설정하고 Is Trigger 속성을 활성화.

05 NPC 오브젝트의 Tag 속성을 Enemy로 설정.

06 NPC 오브젝트가 지정된 경로를 따라 목표 지점으로 이동하는 게임 로직을 구현할 C# 스크립트 파일을 생성하고 이름을 'EnemyMove'로 지정.

2. 웨이브 로직과 적 생성 및 이동

◆적의 이동 로직 구현

07 EnemyMove.cs 파일 작성

코드 14-3 적의 이동 처리 EnemyMove.cs

```
01  using UnityEngine;
02  using UnityEngine.AI;
03
04  public class EnemyMove : MonoBehaviour
05  {
06      public  Transform endObject;
07      public  float arrivalDistance = 0.1f;
08      private NavMeshAgent enemyAgent;
09      private EnemyMgr enemyMgr;
10      private float velocityThreshold = 0.1f;
11
```

03 2D NavMesh 기반 길 찾기와 적 NPC 생성 및 파괴

```
12      public void SetEnemeyMgr(EnemyMgr enemyMgr)
13      {
14          this.enemyMgr = enemyMgr;
15      }
16
17      void Start()
18      {
19          enemyAgent = GetComponent<NavMeshAgent>();
20          if (enemyAgent == null)
21          {
22              Debug.LogError("emeny Agent is None !");
23              return;
24          }
25          enemyAgent.updateRotation = false;
26          enemyAgent.updateUpAxis = false;
27          if (endObject == null)
28          {
29              Debug.LogError("endObject is None !");
30              return;
31          }
32          enemyAgent.SetDestination(endObject.transform.position);
33      }
34
```

2. 웨이브 로직과 적 생성 및 이동

◆적의 이동 로직 구현

```
35      void Update()
36      {
37          if (!enemyAgent.pathPending && enemyAgent.remainingDistance <=
                                                        arrivalDistance &&
             (!enemyAgent.hasPath || enemyAgent.velocity.sqrMagnitude <
                                                      velocityThreshold))
38          {
39              // 도착 시의 처리
40              enemyMgr.DieEnemyAtDestination(gameObject);
41              Destroy(gameObject);
42          }
43      }
44
45      public void SetTarget(Transform target)
46      {
47          if (target == null)
48          {
49              Debug.LogError("target is null!");
50              return;
51          }
```

2. 웨이브 로직과 적 생성 및 이동

◆적의 이동 로직 구현

```
52          endObject = target;
53          if (enemyAgent != null && endObject != null)
54          {
55              enemyAgent.SetDestination(endObject.position);
56          }
57      }
58  }
```

08 NPC 오브젝트에 Enemy.cs와 EnemyMove.cs를 컴포넌트로 추가.

09 Assets 폴더 아래에 Prefabs 폴더를 생성한 후 NPC 오브젝트를 드래그하여 Prefabs 폴더로 옮기고, Scene 뷰에 있는 NPC 오브젝트를 삭제.

2. 웨이브 로직과 적 생성 및 이동

◆웨이브 로직 구현

- 웨이브: 적이 생성되어 맵에 등장하는 패턴
- 웨이브 정보에 스테이지의 총 웨이브 수, 각 웨이브에서 생성할 적의 수와 종류, 생성 간격 등을 정의.

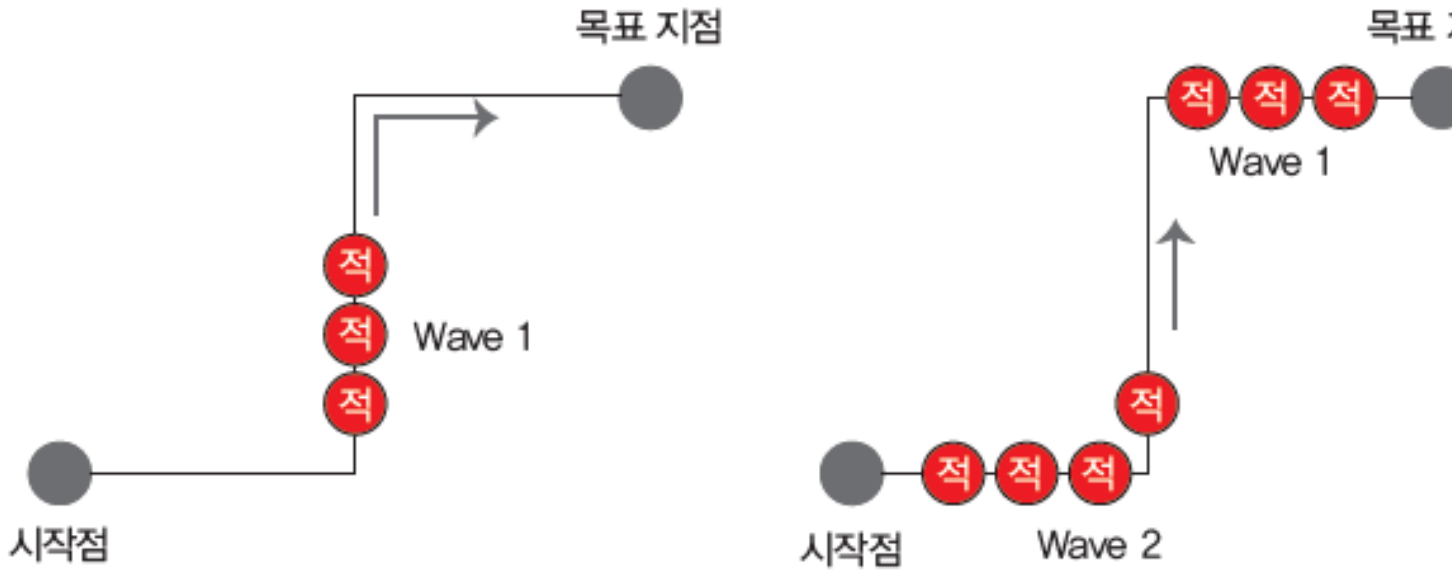


그림 14-29 웨이브에 따른 적의 생성과 움직임

2. 웨이브 로직과 적 생성 및 이동

◆웨이브 로직 구현

01 [Assets]-[Scripts] 폴더에 EnemyMgr.cs 파일을 생성하고 작성.

코드 14-4 적 생성 및 웨이브 처리 EnemyMgr.cs

```
01  using System.Collections.Generic;
02  using UnityEngine;
03
04  [System.Serializable]
05  public class EnemyWaveData
06  {
07      public int enemyCount;              // 이 웨이브에서 생성할 적의 수
08      public float spawnInterval;         // 적 생성 간격(초)
09      public float delayBeforeNextWave;  // 다음 웨이브까지의 대기 시간
10  }
11  public class EnemyMgr : MonoBehaviour
12  {
13      [Header("적 생성을 위한 속성 설정")]
14      public Transform  startPoint;
15      public Transform  endPoint;
16      public GameObject enemyPrefab;     // 생성할 적 프리팹
17      public List<EnemyWaveData> waves = new List<EnemyWaveData>(); // 웨이브 설정
18
```

03 2D NavMesh 기반 길 찾기와 적 NPC 생성 및 파괴

```
19      private bool isStartGame = false;
20      private bool isSpawningWave = false;
21      private bool isWaitNextWave = false;
22      private int  spawnedEnemyNumInWave = 0;
23      private int  curWaveIdx;
24      private List<GameObject> enemyListinCurWave = new List<GameObject>();
25      …생략…
26  private void SpawnEnemyRoutine()
27  {
28      EnemyWaveData currentWave = waves[curWaveIdx];
29      if (curWaveIdx >= waves.Count)
30      {
31
32          CompleteGame();
33          return;
34      }
35      SpawnEnemy();
36      // 현재 Wave의 최대 적 수만큼 생성되지 않았으면 반복 생성
37      if(spawnedEnemyNumInWave+1 < currentWave.enemyCount)
38      {
39          InvokeRepeating("SpawnEnemy",
                            currentWave.spawnInterval,
                            currentWave.spawnInterval
40          );
41      }
42  }
```

03 2D NavMesh 기반 길 찾기와 적 NPC 생성 및 파괴

```
43  private void SpawnEnemy()
44  {
45      EnemyWaveData currentWave = waves[curWaveIdx];
46      GameObject enemyObject = Instantiate(enemyPrefab,
                                 startPoint.position,
                                 Quaternion.identity
47      );
48      EnemyMove enemyMove = enemyObject.GetComponent<EnemyMove>();
49      Enemy enemy = enemyObject.GetComponent<Enemy>();
50      if (enemyMove != null)
51      {
52          enemyMove.SetTarget(endPoint);
53          enemy.SetEnemeyMgr(this);
54          enemyMove.SetEnemeyMgr(this);
55          enemyListinCurWave.Add(enemyObject);
56          Debug.Log($"웨이브 {curWaveIdx} 시작! " + $"({spawnedEneyNumInWave} 적)");
57          spawnedEnemyNumInWave++;
58          if (spawnedEnemyNumInWave >= currentWave.enemyCount) {
59              isSpawningWave = false;
60              CancelInvoke("SpawnEnemy");
61          }
62      }
63  }
64  …생략…
```

2. 웨이브 로직과 적 생성 및 이동

◆웨이브 로직 구현

02 빈 오브젝트를 생성하고 이름을 'CurEnemyMgr'로 지정한 후 EnemyMgr.cs 파일을 컴포넌트로 추가

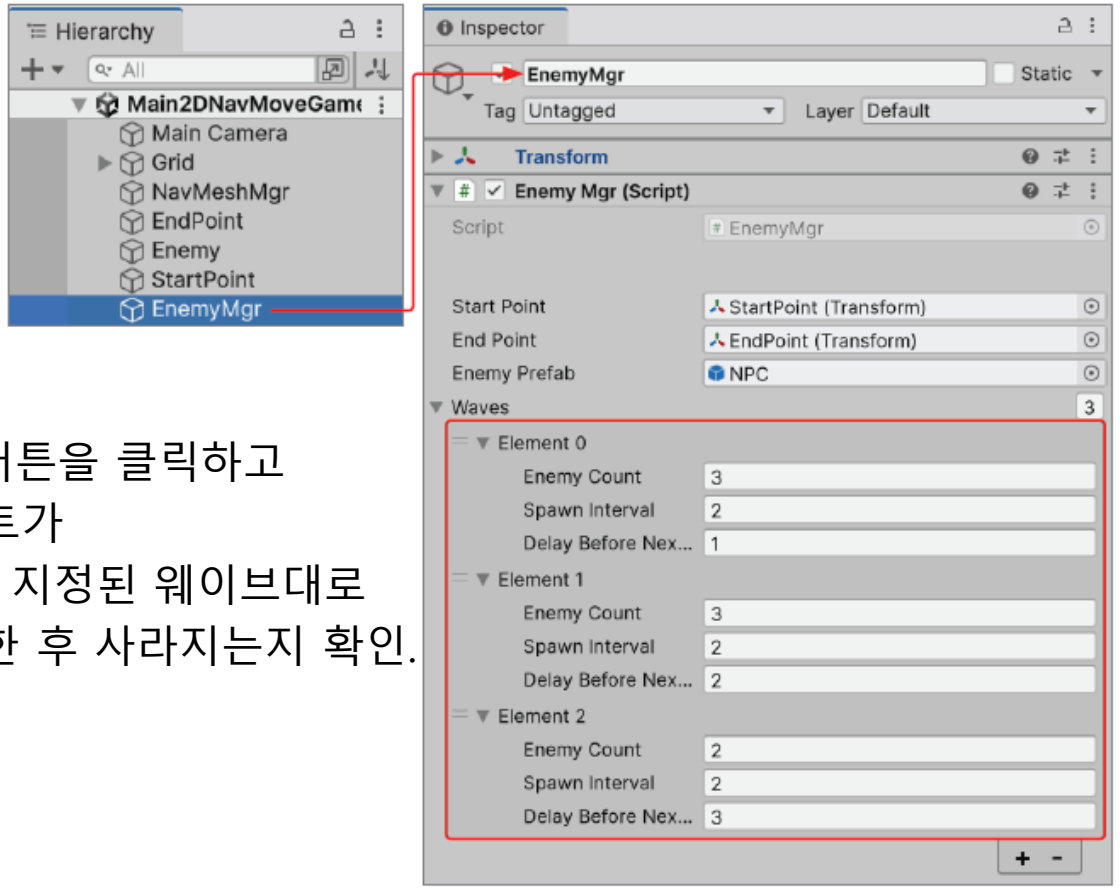
03 StartPoint와 EndPoint 빈 오브젝트를 생성하고, StartPoint의 위치는 (-9.5, -4.5, 0)으로, EndPoint의 위치는 (7, 5.5, 0)으로 설정.

04 StartPoint, EndPoint 오브젝트를 드래그하여 EnemyMgr의 속성에 설정하고, [Assets]-[Prefabs] 폴더에 있는 NPC 오브젝트를 드래그하여 Enemy Prefab에 설정.

2. 웨이브 로직과 적 생성 및 이동

◆웨이브 로직 구현

05 Waves에서 [+]를 클릭하고 웨이브 정보를 생성.



06 유니티 에디터에서 실행 버튼을 클릭하고 Game 뷰에서 Enemy 오브젝트가 StartPoint 위치에서 생성되어 지정된 웨이브대로 EndPoint까지 자동으로 이동한 후 사라지는지 확인.

그림 14-30 EnemyMgr 오브젝트의 설정

04
공격 무기 시스템

1. 무기 생성

- 타일 맵에 타워 배치가 가능한 특수 타일을 지정하고 해당 위치에 공격 타워를 생성.

◆공격 타워 구현

01 Grid 오브젝트의 자식으로 'Building' 타일 맵 생성. Tilemap Renderer 속성에서 Order in Layer를 1로 설정하고, Tilemap 속성에서 Color를 초록색으로 설정.

02 Navigation Modifier를 컴포넌트로 추가한 후 Override Area 속성을 활성화하고 Area를 Not Walkable로 설정. NavMeshMgr 오브젝트의 컴포넌트 Navigation Surface의 <Bake>를 클릭.

03 [2D Object]-[Sprite]-[Circle]을 선택하여 오브젝트 'TowerObj'를 생성. 위치는 (0, 0, 0), Color는 노란색, Order in Layer는 3으로 설정.

1. 무기 생성

◆공격 타워 구현

04 타워는 사정거리 안에서 가장 가까운 적을 공격하는데, 계속 공격하는 것이 아니라 공격 대기 시간만큼 기다렸다가 공격함.

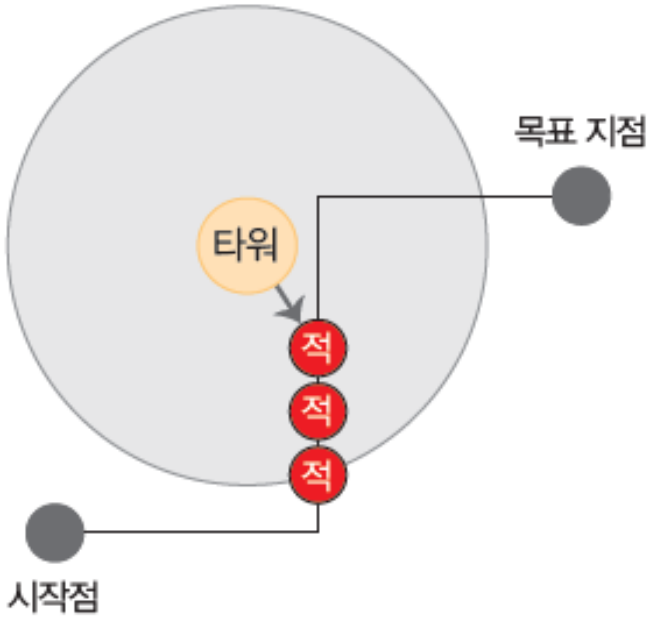


그림 14-31 가장 가까운 적을 공격하는 타워

1. 무기 생성

◆공격 타워 구현

05 C# 스크립트 파일 'Tower'를 생성하고 작성.

코드 14-5 가장 가까운 적을 공격하는 타워의 로직 Tower.cs

```
01  using UnityEngine;
02
03  public class Tower : MonoBehaviour
04  {
```

```
14      void FixedUpdate()
15      {
16          curAttackTime += Time.fixedDeltaTime;
17
18          // 사정거리 안의 적을 탐색하여 가장 가까운 적을 확인
19          if (curAttackTime > waitAttackTime)
20          {
21              if (this.enemyMgr)
22              {
23                  findEnemy = this.enemyMgr.GetClosestEnemy(
                                transform.position,
                                attackRange
24                  );
```

04 공격 무기 시스템

```
25                if (findEnemy)
26                {
27                    GameObject projectileObject = Instantiate(
                                                        projectTile,
                                                        this.transform.position,
                                                        Quaternion.identity
28                    );
29                    Projectile createdProjectile =
                            projectileObject.GetComponent<Projectile>();
30                    createdProjectile.SetDirection(targetDir);
31                    createdProjectile.SetTarget(findEnemy.transform);
32                    createdProjectile.SetEnemeyMgr(this.enemyMgr);
33                }
34            }
35            curAttackTime = 0.0f;
36        }
37    }
38
39    public void SetEnemeyMgr(EnemyMgr enemyMgr)
40    {
41        this.enemyMgr = enemyMgr;
42    }
43  }
```

1. 무기 생성

◆공격 타워 구현

06 Tower.cs 파일을 TowerObj 오브젝트의 컴포넌트로 추가.

07 TowerObj 오브젝트를 드래그하여 [Assets]-[Prefabs] 폴더로 옮기고, Scene 뷰에 있는 TowerObj 오브젝트를 삭제.

2. 공격 시스템과 게임 관리

◆발사체 구현

01 [2D Object]-[Sprite]-[Circle]을 선택하여 'Projectile' 오브젝트를 생성하고, 위치는 (0, 0, 0), 크기는 (0.2, 0.2, 0.2), Color는 파란색, Order in Layer는 3으로 설정.

02 Projectile 오브젝트에 Circle Collider 2D를 컴포넌트로 추가한 후 Radius를 0.2로 설정하고 Is Trigger 속성을 활성화.

03 Projectile 오브젝트에 Rigidbody 2D 컴포넌트를 추가하고, 중력이 적용되지 않도록 Gravity Scale을 0으로 설정.

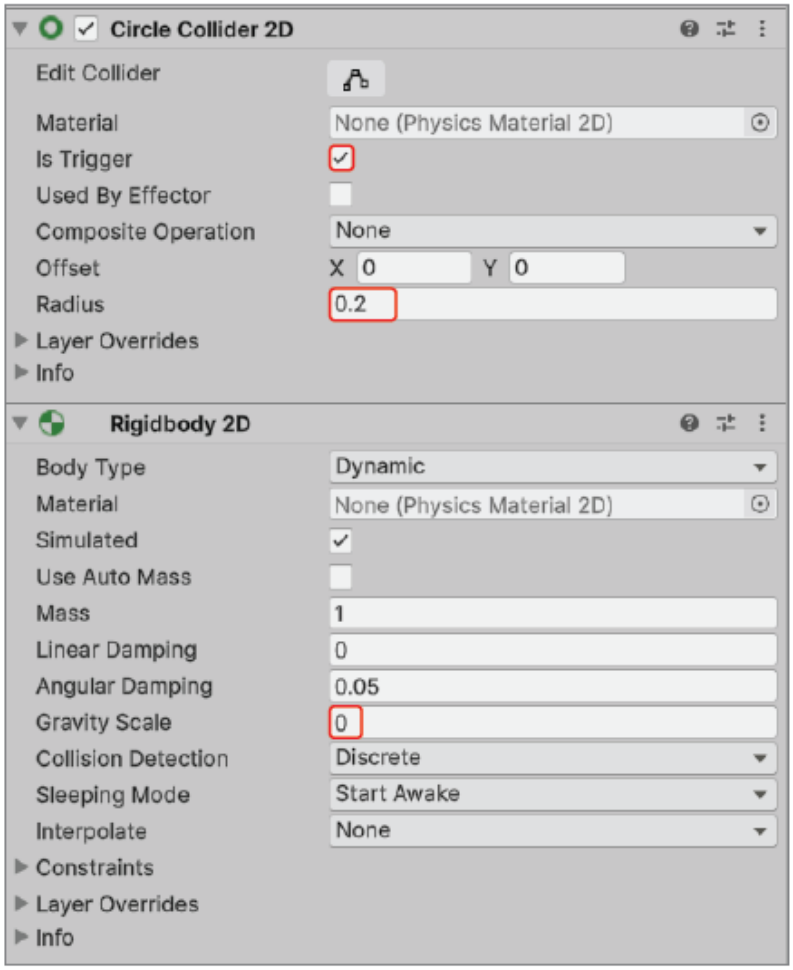


그림 14-32 Projectile의 Circle Collider와 Rigidbody 2D 설정

2. 공격 시스템과 게임 관리

◆발사체 구현

04 C# 스크립트 파일 'Projectile'을 생성하고 작성.

코드 14-6 발사체의 기능 구현 Projectile.cs

```
01  using UnityEngine;
02
03  public class Projectile : MonoBehaviour
04  {
05      [Header("발사체 설정")]
06      public float moveSpeed = 5.0f;
07      public float attackVal = 100.0f;
08      public float lifeTime = 5f;      // 생존 시간(초)
09
10      private Vector3 moveDirection = Vector3.zero;
11      private Vector3 targetPosition; // 목표 위치 저장
12      private Transform targetObj;
13      private float errorDistance = 0.1f;
14      private EnemyMgr enemyMgr;
15
```

04 공격 무기 시스템

```
16      …생략…
17      private void OnTriggerEnter2D(Collider2D collision)
18      {
19          // 충돌한 오브젝트 확인
20          if (collision.CompareTag("Enemy"))
21          {
22              Enemy enemy = collision.gameObject.GetComponent<Enemy>();
23              if (enemy != null)
24              {
25                  enemy.OnDamage(attackVal); // 함수명 수정
26                  Destroy(gameObject); // 오브젝트 삭제로 수정
27              }
28          }
29      }
30      …생략…
31  }
```

05 Projectile.cs 파일을 Projectile 오브젝트의 컴포넌트로 추가.

06 Projectile 오브젝트를 드래그하여 [Assets]-[Prefabs] 폴더로 옮기고, Scene 뷰에 있는 Projectile 오브젝트를 삭제.

07 Projectile 프리팹을 드래그하여 Prefabs 폴더에 있는 Tower 프리팹의 projectTile 속성에 할당.

2. 공격 시스템과 게임 관리

◆타워 생성 관리

01 오브젝트 'CurTowerMgr'을 생성. C# 스크립트 파일 'TowerMgr'을 생성.

코드 14-7 타워 생성 관리 TowerMgr.cs

```
01  using System.Collections.Generic;
02  using UnityEngine;
03  using UnityEngine.InputSystem;
04  using UnityEngine.Tilemaps;
05
06  public class TowerMgr : MonoBehaviour
07  {
08      public Tilemap    buildTileMap;
09      public GameObject buildTower;
10      public EnemyMgr   enemyMgr;
11      public GameMgr    gameMgr;
12      public float      towerPrice = 100.0f;
13
14      // 타일 위에 배치된 타워의 정보 관리
15      private Dictionary<Vector3Int, GameObject> CanBuildTower =
                    new Dictionary<Vector3Int, GameObject>();
16      private Camera mainCamera;
17      private int towerCounter = 0; // 타워 번호 카운팅
```

04 공격 무기 시스템

```
18
19      …생략…
20      void BuildTower()
21      {
22          Vector2 mousePos = Mouse.current.position.ReadValue();
23          Vector3 mouseWorldPos = mainCamera.ScreenToWorldPoint(
                    new Vector3(mousePos.x, mousePos.y,
                                mainCamera.nearClipPlane // 적절한 Z 거리
                    )
24          );
25          Vector3Int cellPosition = buildTileMap.WorldToCell(mouseWorldPos);
26          // 해당 위치에 타일이 있는지 확인
27          if (CanBuildTower.ContainsKey(cellPosition))
28              return;
29          TileBase subTile = buildTileMap.GetTile(cellPosition);
30
31          if (subTile && checkGold(towerPrice))
32          {
33              Vector3 worldPosition = buildTileMap.CellToWorld(cellPosition);
34              Vector3 centerPosition = worldPosition + buildTileMap.cellSize * 0.5f;
35              GameObject createdTower = Instantiate(buildTower, centerPosition,
                                          Quaternion.identity
36              );
```

04 공격 무기 시스템

```
37              createdTower.name = "BasicTower_Pos("+cellPosition.x+","+cellPosition.y+
                                    ")_ID" + towerCounter;
38              CanBuildTower[cellPosition] = createdTower;
39              createdTower.GetComponent<Tower>().SetEnemeyMgr(enemyMgr);
40              // 생성된 타워를 TowerMgr의 자식 노드로 설정
41              createdTower.transform.SetParent(GetComponent<Transform>());
42          }
43      }
44      …생략…
45  }
```

02 TowerMgr.cs 파일을 CurTowerMgr 오브젝트의 컴포넌트로 추가.

03 TowerMgr 컴포넌트의 속성인 Build Tile Map에는 Building 오브젝트를 드래그하여 설정, Build Tower에는 [Assets]-[Prefabs]에서 TowerObj 프리팹을 드래그하여 설정, EnemyMgr에는 CurEnemyMgr 오브젝트를 드래그하여 설정.

2. 공격 시스템과 게임 관리

◆재화와 HP 관리

01 GameMgr.cs 파일과 CurGameMgr 오브젝트를 생성한 후 GameMgr.cs를 CurGameMgr 오브젝트의 컴포넌트로 추가하고 작성.

코드 14-8 재화와 HP 관리 GameMgr.cs

```
01  using UnityEngine;
02  public class GameMgr : MonoBehaviour
03  {
04      [Header("게임 설정")]
05      public float curGold = 300;
06      public float curHP = 100;
07
08      public bool checkGold(float price)
09      {
10          return this.curGold >= price ? true : false;
11      }
12      public void consumeGold(float price)
13      {
14          this.curGold -= price;
15      }
```

04 공격 무기 시스템

```
16      public void damageHP(float damage)
17      {
18          this.curHP -= damage;
19          if (this.curHP <= 0)
20          {
21              this.GameOver();
22          }
23      }
24      private void GameOver()
25      {
26          // GameOver 처리
27          Debug.Log("Game Over");
28      }
29  }
```

02 CurGameMgr 오브젝트를 드래그하여 TowerMgr 컴포넌트의 속성인 GameMgr에 설정

유니티 에디터에서 실행 버튼을 클릭하면 Game 뷰에서 적이 생성되고, 타일 맵에서 타워 생성이 가능한 타일을 클릭했을 때 타워가 생성되어 적을 공격함.

Thank You!

Copyright© 2026 Hanbit Academy, Inc.
All rights reserved.